Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2026-2027
Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

LogInsight: Real-Time Log Intelligence and Incident Investigation Platform

KARKALA SHIVA REDDY	**2520030105**
PARIPALLI NAVADEEP	**2520030196**

COURSE INSTRUCTOR

Dr. J Sirisha Devi
Professor
Department of Computer Science and Engineering

LogInsight: Real-Time Log Intelligence and Incident Investigation Platform

LogInsight is a full-stack log investigation platform that applies classical data structures and algorithms to the problem of explaining a distributed outage. The engineering problem it addresses is specific: during an incident, the log lines that explain the failure are spread across services, arrive interleaved and out of order, and must be queried far faster than a human can read them. The platform ingests a log corpus, streams it to a browser client, detects elevated-error windows automatically, and presents an investigation surface in which every number on screen can be traced back to the algorithm that produced it.

1.1 The case study

A production outage in a microservice estate is rarely explained by a single log line. In the scenario modelled by this project, a payment provider begins returning 502 responses. The failure first appears as one ERROR line in the payment service. Within minutes it propagates: the order service retries, the API gateway begins returning 504, the notification service fails to deliver confirmations, and customers see failed checkouts. The signal is present in the data from the first moment, but it is diluted across roughly fourteen thousand records spanning eight services. An engineer scrolling a flat log view will not find it.

The naive approach is to search. A developer types a phrase such as "connection refused" into a search box and waits for a substring scan to finish. Two problems appear. First, a naive substring search is O(nm) in the length of the text and the pattern, so rescanning the corpus on every keystroke is wasteful and, in the worst case, quadratic in the pattern. Second, and more seriously, the engineer does not know the phrase to search for. The error message that matters is not the one they remember; it is the one that actually occurred.

LogInsight addresses both problems with classical algorithms, and it makes each algorithm's contribution visible rather than implicit. The Knuth-Morris-Pratt matcher gives linear-time exact substring search with no backtracking. The Aho-Corasick automaton tests an entire set of candidate error signatures against each incoming record in a single pass, which is what allows the server to label a generated event with the signature that caused it. The Levenshtein dynamic-programming distance powers a "did you mean" suggestion, so a mistyped or partial query still finds the right corpus. A breadth-first traversal of the observed service graph computes blast radius: the services reachable downstream of the failing one. A fixed-window aggregation with a baseline-relative threshold decides which windows are anomalous.

1.2 System architecture

The system has three layers. The data layer is a deterministic in-memory dataset of fourteen thousand log events, generated by a seeded corpus generator so that every run of the project is reproducible. The algorithm layer is a set of pure Java implementations under the package com.loginsight.dsa, organised as eight packages: string, dp, graph, flow, approximation, randomized, parallel and common. The common package holds hand-written queue and stack structures, so the parallel implementations are built on structures written for this project rather than on a library collection. The service layer exposes those algorithms through REST and Server-Sent Events endpoints, and owns the dataset lifecycle, the incident detector and the deterministic live simulation.

The presentation layer is a React 18 and TypeScript single-page application. Navigation is grouped into five primary Workspace entries (Overview, Logs, Incidents, Services, Analytics) and a collapsible Advanced group holding the remaining fourteen specialist tools; no route was removed, so the whole workspace stays reachable from the sidebar or from the command palette. The application renders an overview command center, a scenario lab, a live monitor, a bounded dataset replay, an incident workbench, an algorithm catalogue with traceable runs, and service views backed by a two-dimensional SVG topology map with an optional lazy-loaded WebGL scene. Server-Sent Events carry each telemetry frame, and a single provider owns the stream so that the overview and the monitor cannot display contradictory numbers.

1.3 Honesty constraints

Two constraints shape the design, and both are enforced in the user interface rather than only in documentation. The first is provenance. The platform distinguishes dataset replay from server-generated simulation at every surface, and simulation surfaces are labelled as generated traffic rather than captured telemetry. Topology surfaces state whether their edges are observed request-trail adjacency or a scenario's declared dependency graph. The second is epistemic restraint: health bands are fixed heuristic thresholds over a rolling window, and the interface describes them as thresholds rather than as a learned model. Incidents are elevated-error windows with no confidence score, because none is computed, and blast radius is described as graph reachability rather than as a confirmed root cause.

1.4 Contribution and verification

The verifiable outcome of the project is a test suite and a green build. The backend contains 910 automated tests and the frontend contains 72 tests across 20 files; both suites run on every push to the main branch through continuous integration, which also runs the TypeScript compiler, the production build and an eighteen-workflow Playwright suite against the real application. The interface was additionally verified in a real browser: no console errors, no horizontal overflow at six viewport widths from 1440x900 down to 360x800, a measured contrast pass with zero failures, keyboard focus indicators on every interactive element, and a runtime stability pass confirming that repeated switching between the two-dimensional and three-dimensional topology,

repeated start and stop of the simulation stream, and repeated route changes produce no DOM growth, no duplicated WebGL renderer and no memory growth.

ALGORITHM / PSEUDO CODE

This section states the algorithms that run inside the product pipeline, as distinct from the additional algorithms exposed only through the Algorithm Lab for study. For each, the problem, the approach, the pseudocode and the complexity are given, followed by the role the implementation plays in the application.

2.1 Knuth-Morris-Pratt substring search (production)

Problem. Given a corpus text T of length n and a single phrase P of length m, find every occurrence of P in T. A naive scan compares P against each of the n starting positions and costs O(nm). Because a log query is re-evaluated on every keystroke, this cost is paid repeatedly.

Approach. Build the failure function (prefix table) for the pattern once. The prefix function pi[i] stores the length of the longest proper prefix of P[0..i] that is also a suffix of it. During scanning, a mismatch does not restart the comparison at the next character; it falls back along the already-matched prefix using pi, so the text pointer never moves backwards.

Pseudocode.

```
BUILD-FAILURE(P, m):
    pi[0] = 0; k = 0
    for i = 1 to m - 1:
        while k > 0 and P[i] != P[k]:
            k = pi[k - 1]                # fall back along the matched prefix
        if P[i] == P[k]: k = k + 1
        pi[i] = k
    return pi

KMP-SEARCH(T, P):
    m = length(P)
    if m = 0: return empty
    pi = BUILD-FAILURE(P, m); matches = []; k = 0
    for i = 0 to length(T) - 1:
        while k > 0 and T[i] != P[k]:
            k = pi[k - 1]                # no rewind of i: linear overall
        if T[i] == P[k]: k = k + 1
        if k = m:
            matches.append(i - m + 1)
            k = pi[k - 1]                # allow overlapping matches
    return matches
```

Complexity. Building the prefix function is O(m); scanning is O(n) because the text pointer advances monotonically and the total fall-back work telescopes to O(n). Memory is O(m).

Role in the application. com.loginsight.dsa.string.KMPMatcher is the default matcher for phrase search in the log search service, is used by the live simulation detector to locate the dominant error signature inside a generated message, and backs the traceable KMP endpoint.

2.2 Aho-Corasick multi-pattern search (production)

Problem. The detector must test many candidate error signatures at once against every incoming record. Running one independent search per signature multiplies the cost by the number of signatures.

Approach. Build a trie of the signature set, complete it with failure links to states of the suffix automaton, and annotate each state with the patterns that end there. Scanning then reports every pattern occurring in the text during a single left-to-right pass using the current state's output list.

Pseudocode.

```
BUILD-TRIE(patterns):
    root = new node; for each pattern s:
        cur = root
        for c in s:
            if cur.next[c] is undefined: cur.next[c] = new node
            cur = cur.next[c]
        cur.output.add(s)                # pattern terminates at this state
    return root

BUILD-FAILURE(root):                             # BFS over trie depth
    queue = [root]
    while queue is not empty:
        for c in alphabet:
            if root.next[c] is undefined: root.next[c] = root        # goto root
            else: fail[root.next[c]] = root; queue.append(root.next[c])
    while queue is not empty:
        r = queue.pop_front()
        for c in alphabet:
            u = r.next[c]
            if u is undefined: r.next[c] = fail[r].next[c]           # inherit goto
            else:
                fail[u] = fail[r].next[c]
                fail[u].output = fail[u].output union fail[fail[u]].output
                queue.append(u)

SCAN(root, text):                     # one pass reports every signature
    state = root; hits = []
    for c in text:
        state = state.next[c]
        for pattern in state.output: hits.add(pattern)
    return hits
```

Complexity. Construction is O(total signature length plus states times alphabet). Scanning is O(n) in the text length regardless of how many patterns match, which is the property that makes it the right choice for per-record signature labelling.

Role in the application. com.loginsight.dsa.string.aho.AhoCorasick is built once per simulation scenario from that scenario's declared error signatures and reused across frames, and it backs the traceable multi-pattern endpoint.

2.3 Levenshtein edit distance and suggestion ranking (production)

Problem. An operator who does not know the exact error phrase still needs to find the right records. Fuzzy matching requires a distance between the typed query and each candidate message.

Approach. Fill a dynamic-programming table where cell D[i][j] is the edit distance between the first i characters of A and the first j characters of B. The recurrence takes the cheapest of three moves: delete, insert, or substitute. Retaining the table also allows the

optimal edit script to be reconstructed by backtracking, which the interface displays as evidence.

Pseudocode.

```
LEVENSHTEIN(A, B):                    # D[0][j] = j, D[i][0] = i
    n = length(A); m = length(B)
    D[0][j] = j for j = 0..m;   D[i][0] = i for i = 0..n
    for i = 1 to n:
        for j = 1 to m:
            cost = 0 if A[i] = B[j] else 1
            D[i][j] = min( D[i-1][j] + 1,          # deletion
                           D[i][j-1] + 1,          # insertion
                           D[i-1][j-1] + cost )    # substitution
    return D[n][m], D

SUGGEST(query, distinctMessages, maxDistance):
    best = null; bestDistance = infinity; freq = {}
    for m in distinctMessages:
        freq[m] = freq.get(m, 0) + 1
        d = LEVENSHTEIN(query, m)
        if d <= maxDistance and (d < bestDistance
                                 or (d = bestDistance and freq[m] > freq[best])):
            best = m; bestDistance = d
    return best
```

Complexity. O(nm) time and O(nm) space for the full table; the distance alone needs O(min(n, m)) space with a rolling row. The suggestion pass is linear in the number of distinct messages and is bounded by a maximum-distance cutoff.

Role in the application. com.loginsight.dsa.dp.editdistance.LevenshteinDistance ranks the "did you mean" suggestion offered by the log search service, and powers the bounded fuzzy search engine and the traceable edit-distance endpoint. The Damerau-Levenshtein variant adds the adjacent transposition move over the same table shape.

2.4 Breadth-first traversal for blast radius (production)

Problem. When an incident names a failing service, the operator needs the set of services that can be affected downstream. This is a reachability question over a directed service graph.

Pseudocode.

```
REACHABLE-FROM(graph, start):           # BFS over outbound edges
    visited = new set;  queue = new deque
    queue.add(start);  visited.add(start)
    while queue is not empty:
        node = queue.removeFirst()
        for next in graph.outbound(node):
            if next not in visited:     # mark on enqueue: no duplicates
                visited.add(next); queue.addLast(next)
    return visited
```

Complexity. O(V + E) time and O(V) space, because each vertex is enqueued exactly once and each edge is inspected once. Marking on enqueue rather than on dequeue is what prevents repeated enqueues in cyclic graphs.

Role in the application. com.loginsight.graph.ServiceDependencyGraph supplies the blast-radius set that the incident workbench highlights in the topology. The interface labels this as a traversal over observed adjacency and explicitly not as a causal claim.

2.5 Fixed-window anomaly detection (production)

Problem. Marking an incident requires deciding which part of a timeline is anomalous, without assuming the corpus has a globally uniform error rate.

Approach. Bucket events into fixed five-minute windows across the dataset span. Compute the mean error count per window as the baseline, then flag a window whose error count reaches max(3, 3 x baseline). Finally merge consecutive, or one-window-apart, flagged windows into a single incident so that one failure does not produce several alerts. The health band shown in the interface applies the same relative idea continuously: healthy below a five percent window error rate, watch below ten percent, elevated at or above it.

Pseudocode.

```
DETECT-INCIDENTS(events, window = 5 min, factor = 3.0):
    buckets = array of size windowCount
    for each event: buckets[(event.time - start) / window] += 1
    for each bucket b: errors[b] = count of ERROR events in b
    baseline = mean(errors)
    threshold = max(3, factor * baseline)   # absolute floor of 3
    flagged = { b : errors[b] >= threshold }
    merge flagged windows that are consecutive or one window apart -> incidents
    for each incident: attach services, severity and evidence rows
```

Complexity. One pass to bucket in O(n), a linear scan for the baseline and the flags, and a linear merge over the flagged set, giving O(n + W) for n events and W windows. The window count is capped so a pathologically long span cannot allocate an unbounded array.

2.6 Additional algorithms exposed for study

The Algorithm Lab exposes the wider classical repertoire required by the course as traceable, individually runnable implementations with complexity reported alongside each result. The catalogue returned by the backend currently holds 42 algorithms across six modules: 9 in Strings, 12 in Dynamic Programming, 6 in Graph and Flow, 7 in Approximation, 5 in Randomized and 3 in Parallel. Of these, 36 are reachable through a canonical or a trace endpoint, dispatched across 35 distinct keys, and 13 are additionally traceable. The repertoire covers the Z algorithm and Rabin-Karp matching; suffix-array construction with a Kasai longest-common-prefix table; Levenshtein, Damerau-Levenshtein and weighted edit distance, together with Needleman-Wunsch and Smith-Waterman alignment; bitmask travelling-salesman and Hamiltonian-path search; interval dynamic programming for matrix-chain order and optimal binary search trees; tree dynamic programming for diameter, rerooting and sum-over-subsets; Ford-Fulkerson, Edmonds-Karp and Dinic flow with minimum cut, bipartite matching and min-cost max flow; vertex-cover kernelization and approximation with parameterized knapsack; randomized quicksort, Miller-Rabin primality, universal and two-level perfect hashing and reservoir sampling; and parallel reduce, prefix scan and sample sort with work and span analysis. These are deliberately separated from the production pipeline: the System page reports which algorithms are exposed through an API and which are traceable, so the distinction is visible rather than implied.

CODE

The project is implemented as a Java 21 Spring Boot 3.5 backend and a React 18 with TypeScript frontend. The complete source is in the project repository. The excerpts below are the production implementations of the algorithms described in the previous section, reproduced from the repository so that the code and the pseudocode can be compared directly.

3.1 Knuth-Morris-Pratt matcher

File: backend/src/main/java/com/loginsight/dsa/string/KMPMatcher.java. The scan loop never rewinds the text cursor; on a mismatch the pattern state is pulled back through the longest-prefix table instead of restarting the comparison, which is what makes the total work linear.

```
public final class KMPMatcher implements StringMatcher {

    @Override
    public StringSearchResult match(String text, String pattern) {
        TextValidator.requireNonNull(text, pattern);
        TextValidator.requireNonEmpty(pattern);
        char[] t = text.toCharArray();
        char[] p = pattern.toCharArray();
        int n = t.length;
        int m = p.length;

        long start = System.nanoTime();
        int[] lps = buildLps(p);
        int[] positions = new int[n];
        int count = 0;
        int i = 0;
        int j = 0;
        while (i < n) {
            if (t[i] == p[j]) { i++; j++; }
            if (j == m) {
                positions[count++] = i - j;
                j = lps[j - 1];          // allow overlapping matches
            } else if (i < n && t[i] != p[j]) {
                if (j != 0) {
                    j = lps[j - 1];      // fall back along the matched prefix
                } else {
                    i++;
                }
            }
        }
        // ... build StringSearchResult with matches, runtime and complexity
```

3.2 Aho-Corasick automaton scan

File: backend/src/main/java/com/loginsight/dsa/string/aho/AhoCorasick.java. One left-to-right pass reports every signature. Dictionary-suffix links walk to output-bearing suffix states so a pattern that is a suffix of another is still reported, and the scan is occurrence-capped so a pathological pattern set cannot build an unbounded result list.

```
public PatternMatch[] search(String text, int maxMatches) {
    if (text == null) throw new IllegalArgumentException("text must not be null");
    if (maxMatches < 0) throw new IllegalArgumentException("maxMatches must be >= 0");
    char[] chars = text.toCharArray();
    int[] ids = new int[16];
    int[] starts = new int[16];
    int matchCount = 0;

    int state = 0;
```

```
        for (int i = 0; i < chars.length; i++) {
            char c = chars[i];
            while (state != 0 && nodes[state].findChild(c) == -1) {
                state = nodes[state].getFail();       // failure link
            }
            int g = nodes[state].findChild(c);
            state = g == -1 ? 0 : g;
            // report this node, then every output-bearing suffix via dict-suffix links
            for (int v = state; v != 0 && v != -1; v = nodes[v].getDictSuffix()) {
                int id = nodes[v].getOutput();
                if (id != -1) {
                    if (matchCount >= maxMatches) {
                        throw new IllegalArgumentException(
                            "the scan reports more than " + maxMatches + " occurrences");
                    }
                    ids = ensureCapacity(ids, matchCount);
                    starts = ensureCapacity(starts, matchCount);
                    ids[matchCount] = id;
                    starts[matchCount] = i - patterns[id].length() + 1;
                    matchCount++;
                }
            }
        }
        PatternMatch[] matches = new PatternMatch[matchCount];
        for (int i = 0; i < matchCount; i++) {
            matches[i] = new PatternMatch(ids[i], patterns[ids[i]], starts[i]);
        }
        return matches;
    }
```

3.3 Breadth-first reachability for blast radius

File: backend/src/main/java/com/loginsight/graph/ServiceDependencyGraph.java.

```
    /** All services reachable from {@code start} following outbound edges (BFS). */
    public List<String> reachableFrom(String start) {
        List<String> visited = new ArrayList<>();
        Set<String> seen = new LinkedHashSet<>();
        ArrayDeque<String> queue = new ArrayDeque<>();
        if (outbound.containsKey(start)) {
            queue.add(start);
            seen.add(start);
        }
        while (!queue.isEmpty()) {
            String current = queue.removeFirst();
            visited.add(current);
            for (String next : outbound.getOrDefault(current, Set.of())) {
                if (seen.add(next)) {
                    queue.addLast(next);
                }
            }
        }
        return visited;
    }
```

3.4 Incident lifecycle state machine

File: backend/src/main/java/com/loginsight/simulation/SimulationIncident.java and IncidentLifecycleStore.java. Incident status moves forward through a fixed state machine. Before this project was corrected, an illegal transition was accepted with HTTP 200 and silently ignored, so a client could believe it had closed an incident that was still open. The guard below rejects the request, the exception carries the incident id and both states so the message cannot be malformed, and the exception handler maps it to HTTP 409 Conflict.

```
    public synchronized boolean canTransitionTo(Status target) {
        return target != null
                && target.ordinal() >= status.ordinal()   // forward only
```

```
            && status != Status.RESOLVED;             // RESOLVED is terminal
}

// IncidentLifecycleStore
public synchronized Optional<SimulationIncident> transition(int id, SimulationIncident.Status
target, Instant at) {
    SimulationIncident incident = incidents.get(id);
    if (incident == null) {
        return Optional.empty();
    }
    if (!incident.canTransitionTo(target)) {
        throw new IllegalLifecycleTransitionException(id, incident.getStatus(), target);
    }
    incident.transitionTo(target, at, false);
    return Optional.of(incident);
}

// The exception carries the id and both states, so the message cannot be
// malformed by a caller concatenating it by hand.
public static final class IllegalLifecycleTransitionException extends RuntimeException {
    public IllegalLifecycleTransitionException(int incidentId,
                                       SimulationIncident.Status current,
                                       SimulationIncident.Status requested) {
        super("Incident " + incidentId + " is " + current
                + "; lifecycle transitions only move forward, so "
                + requested + " is not reachable");
    }
}

// GlobalExceptionHandler
@ExceptionHandler(IncidentLifecycleStore.IllegalLifecycleTransitionException.class)
public ResponseEntity<?> lifecycleConflict(IllegalLifecycleTransitionException e,
HttpServletRequest request) {
    return body(HttpStatus.CONFLICT, "IllegalLifecycleTransitionException", e.getMessage(),
request);
}
```

3.5 Deterministic simulation frame

The live surface is a server-owned deterministic stream rather than recorded traffic. A frame is a pure function of the scenario id, the seed and the tick index, so the same request always produces the same events. The speed setting affects only delivery pacing; it never changes event content.

```
// The frame is reproducible: identical (scenario, seed, tick) always yields
// identical events, regardless of the requested delivery speed.
SimulationFrameDto frame(String scenarioId, long seed, int tick) {
    ScenarioDefinition scenario = catalog.require(scenarioId);
    List<LogEvent> events = generator.eventsFor(scenario, seed, tick);
    for (LogEvent event : events) {
        PatternMatch[] hits = automatonFor(scenario).match(event.getMessage());
        // attach the matched signatures as evidence rows
    }
    List<SimulationIncidentDto> incidents = detector.evaluate(events, tick);
    return new SimulationFrameDto(sessionId, tick, metrics, events,
                                  signals, evidence, health, incidents, topology);
}
```

The complete backend and frontend sources, together with the 910 backend tests and 72 frontend tests, are published in the project repository referenced on the cover page.

RESULTS

All screenshots below were captured from the running application against the final build, with the demo dataset loaded and, where a stream is shown, an active deterministic run. The three-dimensional figure is a real rendered WebGL scene rather than the fallback state. The test and build figures quoted here are those produced by the last continuous-integration run on the main branch.

4.1 Verification summary

Backend: 910 automated tests, 0 failures, 0 errors, 0 skipped; the Spring Boot jar packages successfully and JaCoCo coverage is produced. Frontend: 72 tests across 20 files, 0 failures, with a clean TypeScript compile and a successful production build whose first-load JavaScript is approximately 483 kB across the index, React, motion and icon vendor chunks, with the Three.js bundle deferred until the 3D topology is requested. The dependency audit reports no known vulnerabilities. Continuous integration runs the Maven verify goal, the frontend test suite and the frontend build on every push to the main branch, so these figures are reproducible by anyone who clones the repository.

Browser verification was performed separately, because unit tests do not exercise layout or interaction. Eighteen Playwright workflows run against the live application, and the results were: zero console errors on every route; zero horizontal overflow at 1440x900, 1280x800, 1024x768, 768x1024, 390x844 and 360x800, asserted on both the command center and the service map; a measured contrast pass with zero failures against the required 4.5:1 for body text; a visible keyboard focus indicator on every element reached by tabbing; and a stability pass in which repeated switching between the two-dimensional and three-dimensional topology, repeated start and stop of the live stream and repeated route changes produced no DOM growth, no duplicated WebGL renderer and no memory growth.

4.2 Overview command center

The overview leads with the active data source and selected time window, a backend-derived signal strip, the current incident state, the service topology and the recent event feed. The hero block is deliberately compact: the dataset context, the four headline numbers and the primary actions fit inside the first viewport at desktop width, so the selected-window metrics are readable without scrolling. The main row below is asymmetric by design: telemetry takes the wider column and the incident panel the narrower one, so the two are never given equal weight. The generated simulation is reported in a band beneath the dataset evidence rather than inside the hero, because it is a different source from the loaded dataset and must not be read as part of it.

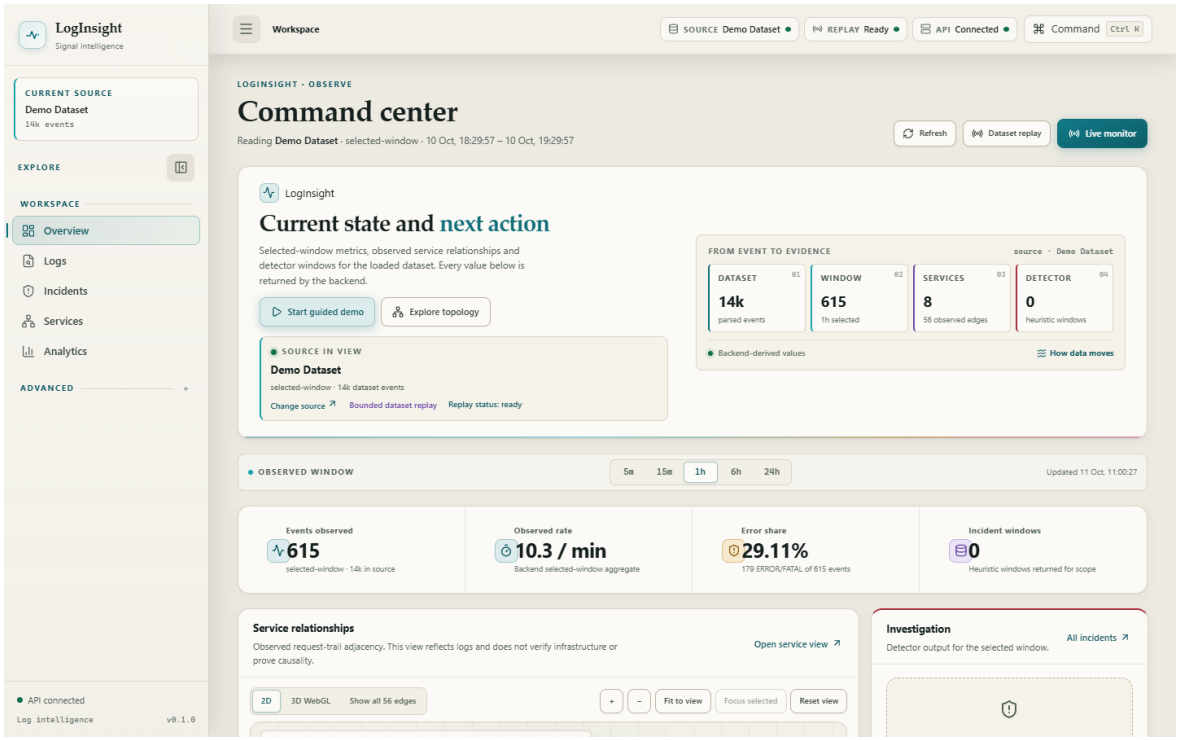


Figure 1. Overview command center with the demo dataset loaded.

4.3 Scenario Lab and deterministic live simulation

The Scenario Lab presents seven declared failure scenarios: Deployment Regression, Traffic Surge, Authentication Burst, Checkout 5xx Cascade, Database Latency Spike, Cache Failure and Payment Timeout. Selecting one shows its title, severity, affected services, error signatures and expected signal. Starting a run opens a Server-Sent Events stream; the tick counter, measured health band, scenario phase, error rate, p95 latency, matched signatures and emerging incidents all advance together, and the evidence rows show which matcher produced each signature. Figure 2 shows a run in progress on the Checkout 5xx Cascade scenario, with the measured band elevated, the phase at peak and one incident already open. Figure 3 shows the catalogue with that same scenario selected before the run begins.

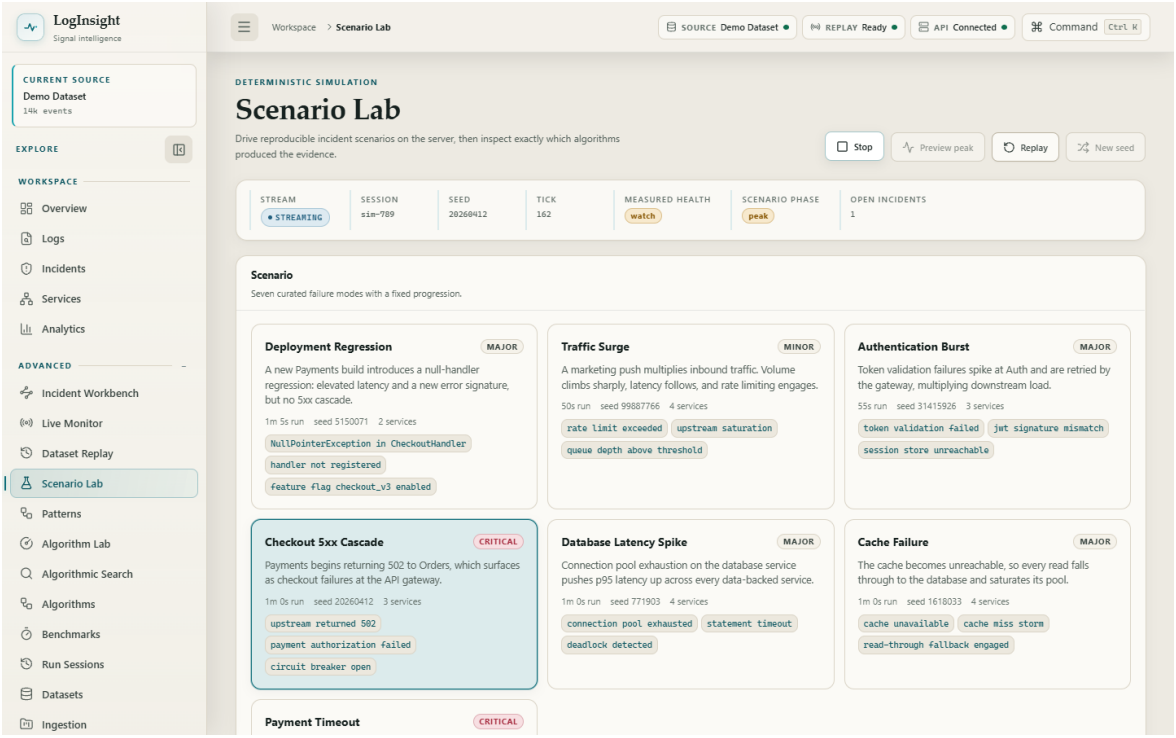


Figure 2. A scenario run in progress on the Scenario Lab.

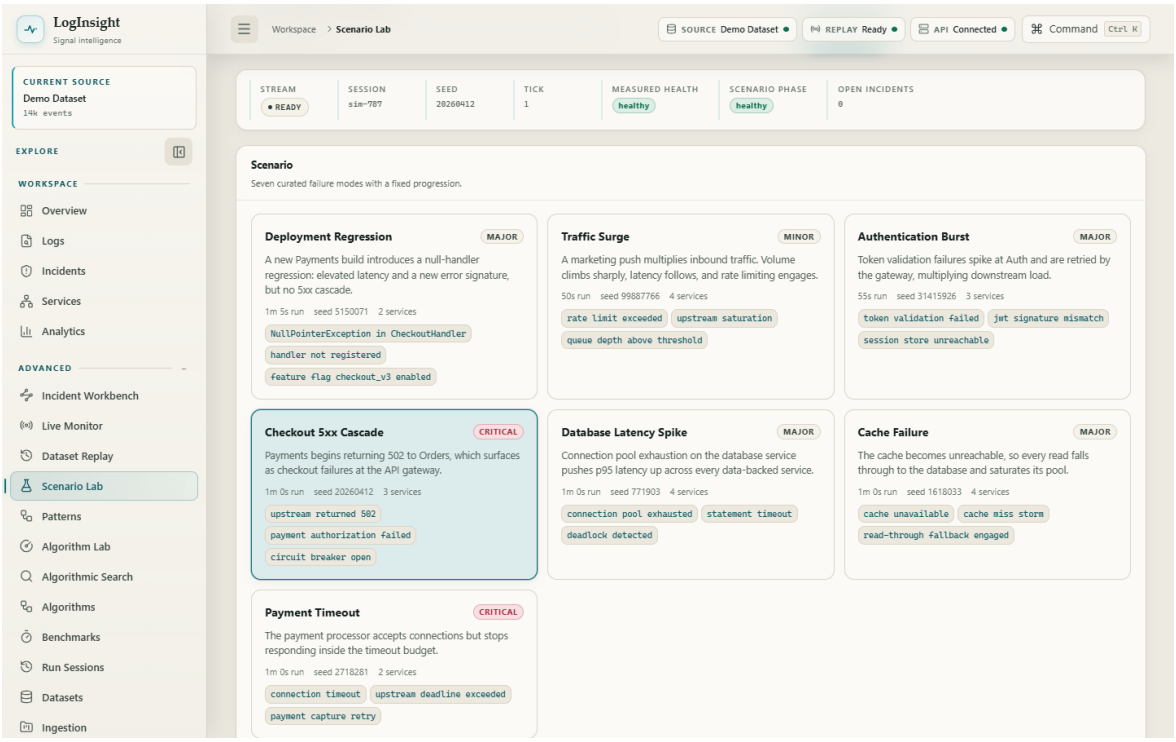


Figure 3. Scenario catalogue and the selected scenario.

4.4 Incident workbench

The workbench is the flagship investigation surface and uses a three-pane layout: an incident navigator at 220 to 280 pixels, the selected incident in the largest centre column, and contextual health and topology at 280 pixels or more. Below 1180 pixels the grid collapses to a single column and the navigator becomes a horizontally scrolling row, so

the incident list stays reachable. The centre column carries the title, severity, lifecycle controls, summary, vertical timeline, signals, evidence rows and correlated events. Lifecycle actions are restricted to forward transitions; an attempt to move backwards or to reopen a closed incident is rejected with HTTP 409 rather than silently accepted.

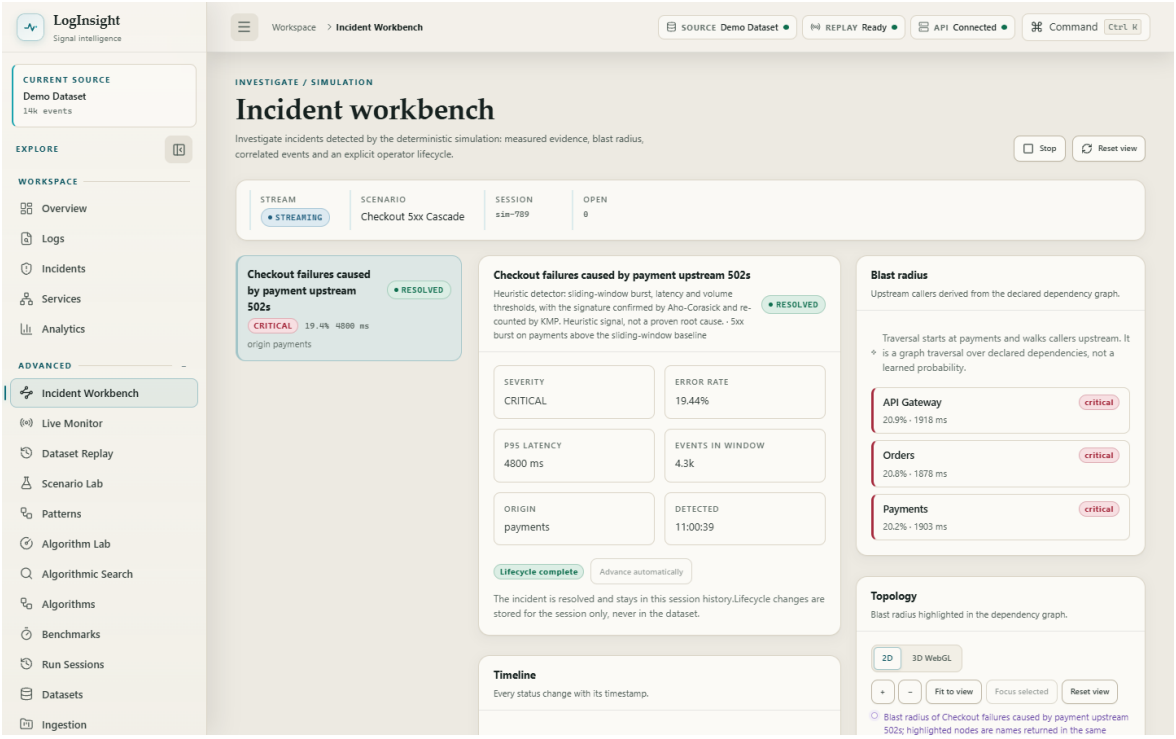


Figure 4. Incident workbench: navigator, detail and context panes.

4.5 Live monitor

The Live Monitor presents the running stream in the order an operator reads it: stream controls, then the hero telemetry state, then the KPI row, then the chart, then topology, then signals and evidence, and finally the event stream. The page therefore progresses from system state to measurement to detection to context. It reads the same Server-Sent Events stream as the incident workbench, so the two surfaces cannot disagree about what is currently happening.

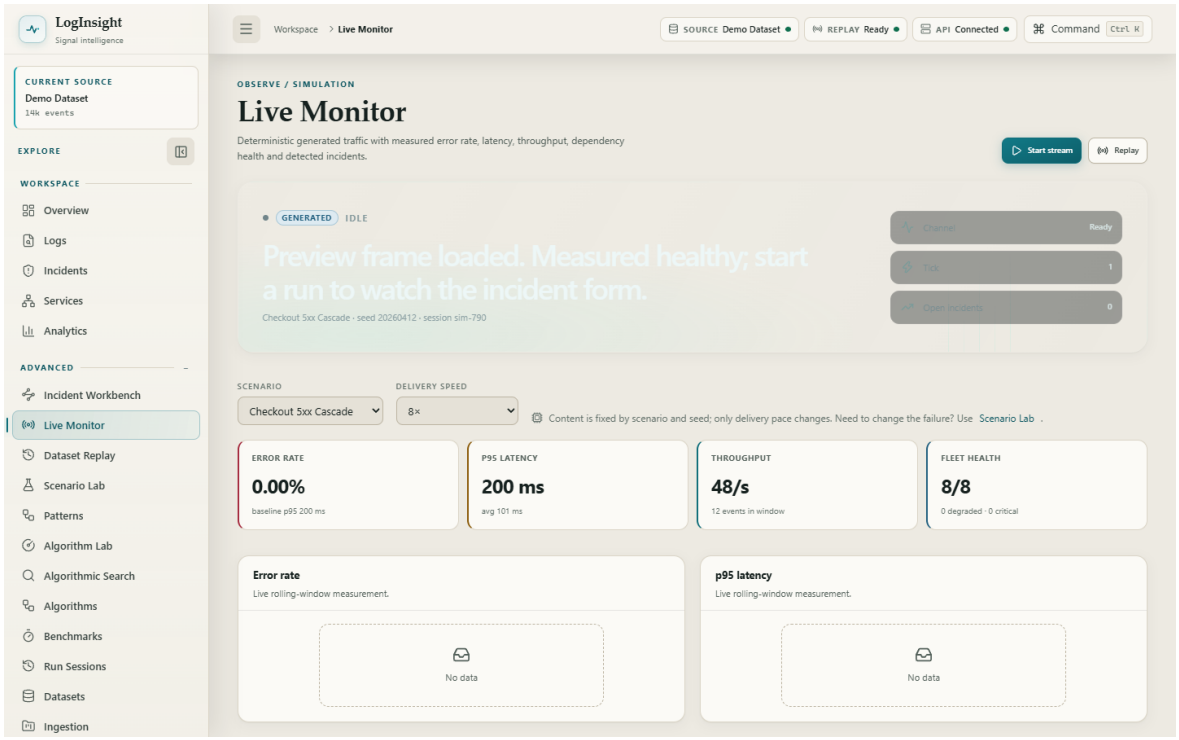


Figure 5. Live Monitor: hero state, metrics, chart, topology and event stream.

4.6 Service topology, two-dimensional and three-dimensional

Both renderers consume the same nodes and edges returned by the dependency API. Node size represents measured event volume and node state represents the measured health band; neither encodes importance or request rate. The two-dimensional view is an accessible SVG map with a paired node and edge list for keyboard and screen-reader users, and it remains fully usable when WebGL is unavailable. The three-dimensional view is a Three.js scene in a separate lazy-loaded bundle, so it is downloaded only when the user selects it. Its lifecycle is the part that had to be built carefully. The scene effect is keyed on a content hash of the graph and the highlighted incidents rather than on the raw arrays, so a parent render with unchanged data no longer tears the scene down and rebuilds it. The context is released explicitly with forceContextLoss on unmount, because dispose alone leaves the underlying WebGL context alive and browsers cap how many may exist at once. A webglcontextrestored listener rebuilds the scene after the browser drops it. The animation loop pauses when an IntersectionObserver reports the panel out of view or a visibilitychange event reports a hidden tab, and the camera does not auto-rotate or auto-zoom. Every renderer, geometry, material, observer and listener resource is disposed on unmount. If a context cannot be created, the panel falls back to the two-dimensional map and offers a Retry WebGL control rather than presenting an empty stage.

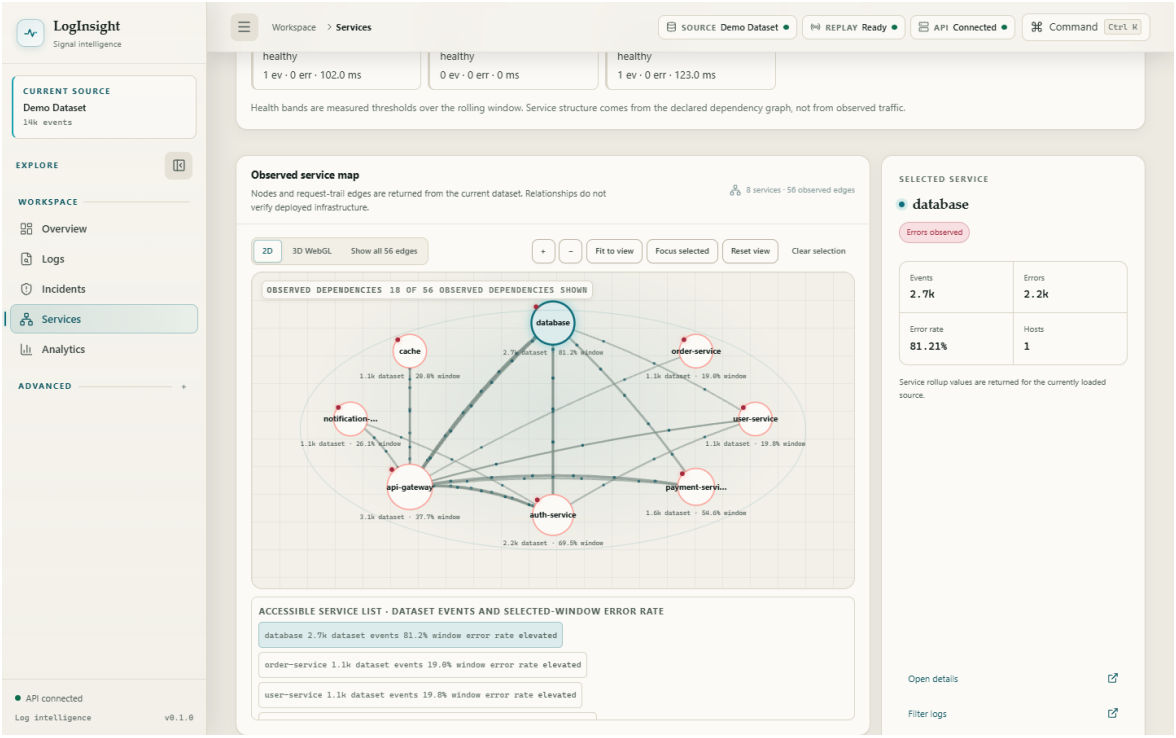


Figure 6. Two-dimensional service topology (accessible SVG map).

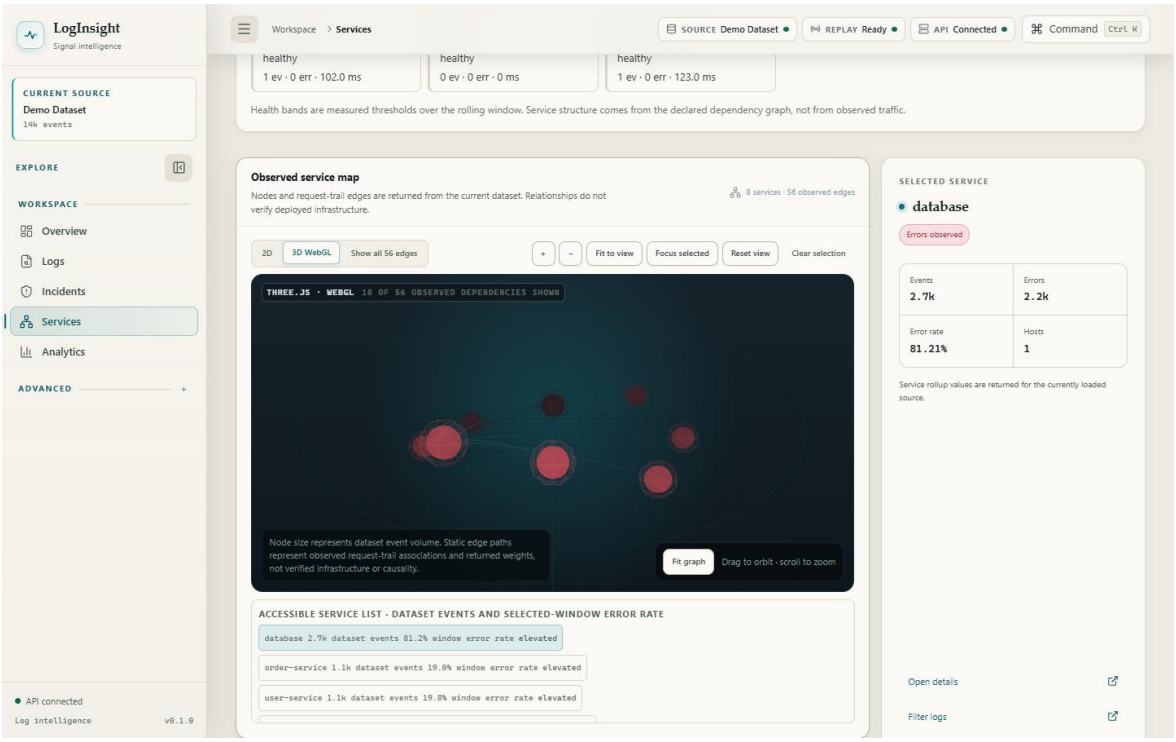


Figure 7. Three-dimensional service topology, rendered as a real WebGL scene from the observed dependency graph.

4.7 Algorithm evidence and dataset analysis

The Algorithm Lab lists each technique with its module, query type, time complexity, space complexity and exposure, and allows a catalogue default input to be run and traced. The backend currently reports 42 algorithms, of which 36 are reachable through a

canonical or a trace endpoint across 35 distinct dispatch keys, and 13 are traceable; the System page reports those counts, which keeps the boundary between the production pipeline and the study catalogue visible rather than implied. Figure 8 shows the catalogue; Figure 9 shows analytics computed over the loaded dataset.

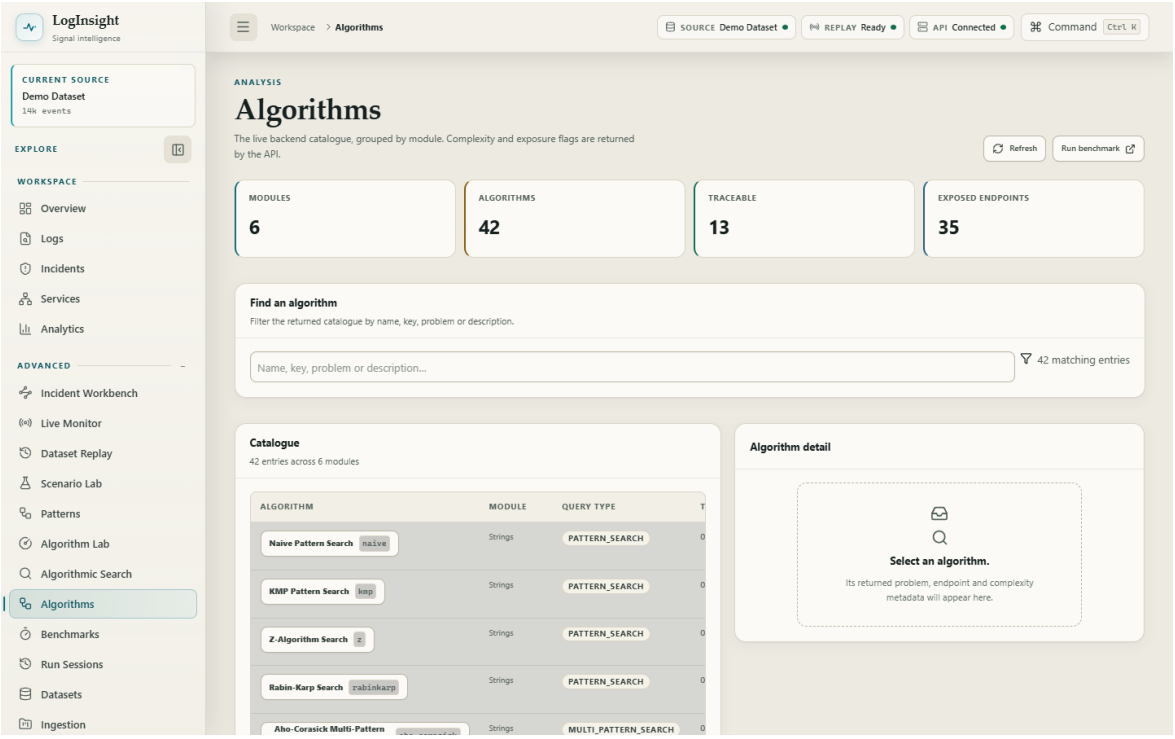


Figure 8. Algorithm catalogue with complexity and exposure.

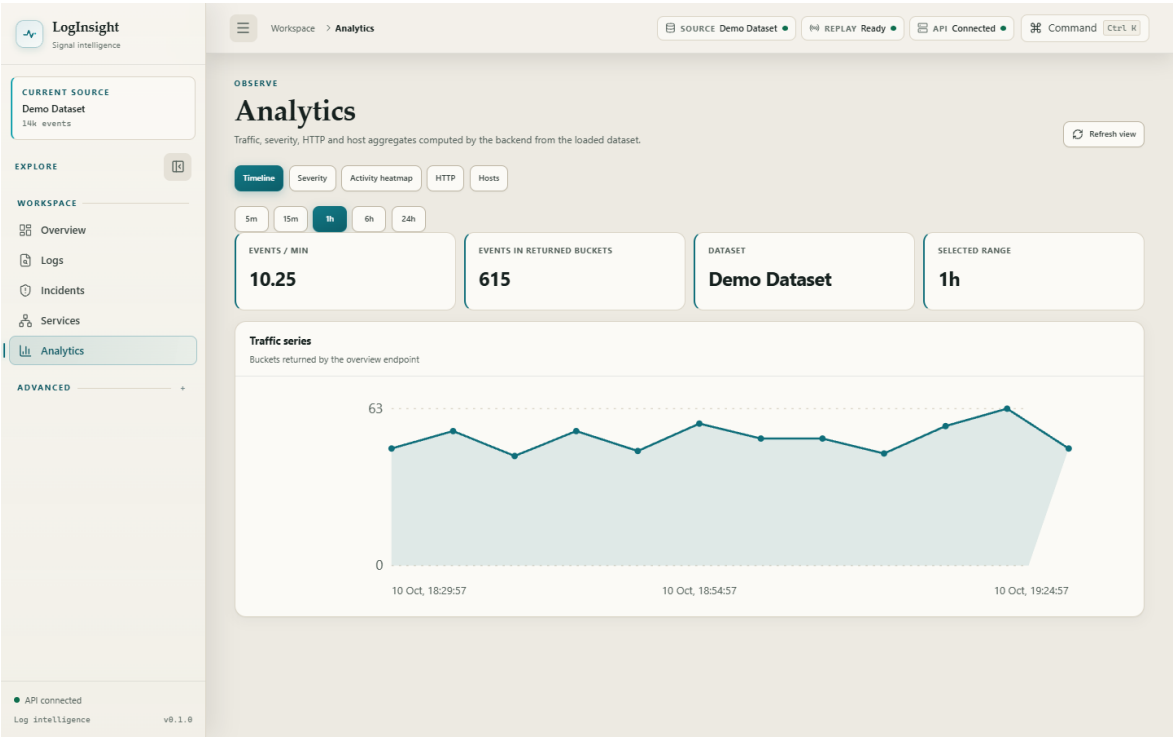


Figure 9. Dataset analytics over the loaded demo corpus.

4.8 Responsive behaviour

The dashboard is restructured rather than scaled down on small viewports. At 1024 pixels the secondary columns are reduced while hierarchy is preserved. At 390 pixels the content becomes a single column in priority order, navigation collapses behind a drawer, tables scroll inside their own container, and interactive targets grow to 32 pixels. The suite asserts this at six widths from 1440x900 down to 360x800, on both the command center and the service map, and no viewport produced horizontal page overflow.

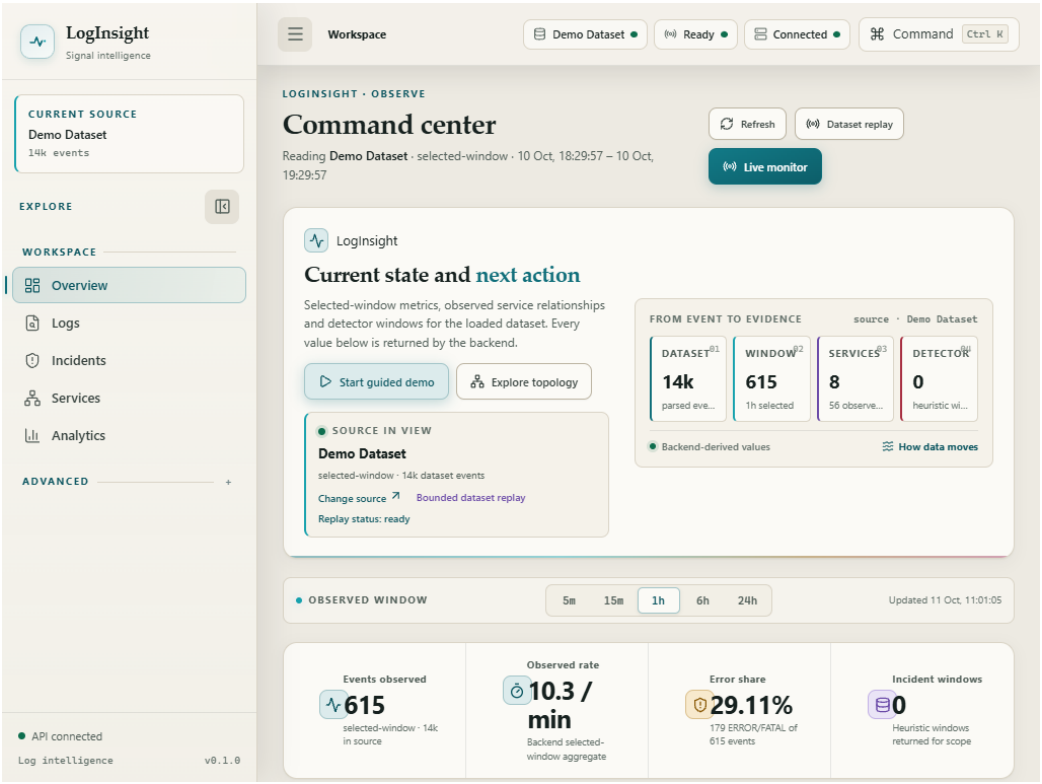


Figure 10. Overview at 1024x768.

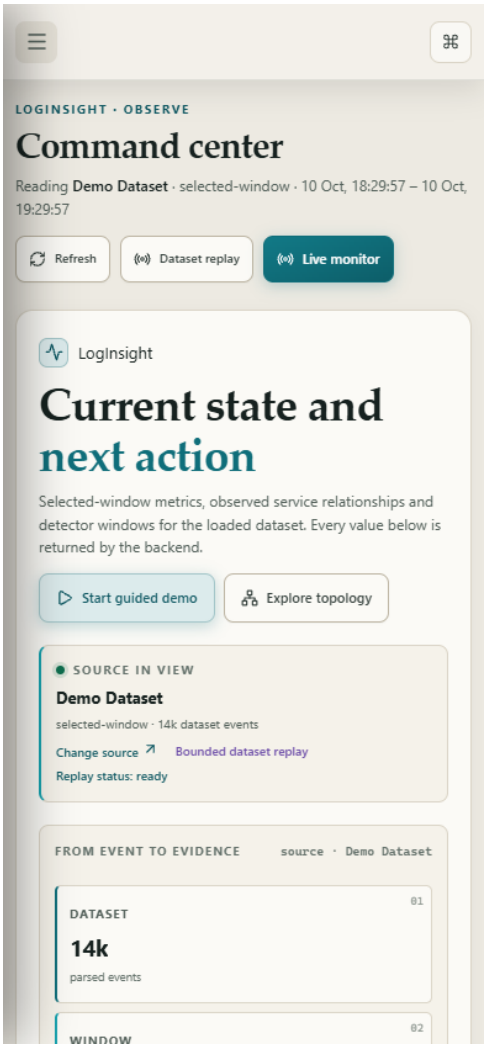


Figure 11. Overview at 390x844.

4.9 Limitations

The platform is an academic project and its limits should be stated plainly. State is held in memory in a single JVM, so there is no durable storage and no multi-node deployment; restarting the backend discards the loaded dataset and any running stream. The simulation generates traffic deterministically and is labelled as generated everywhere it appears, so it is not evidence about any real system. Health bands are fixed heuristic thresholds rather than a trained or validated model, and incidents are elevated-error windows with no confidence score because no confidence score is computed. Run history, event buffers and trace steps are bounded, which protects memory at the cost of history depth. Finally, the Docker Compose files describe a single-node deployment shape and were validated for syntax only, because no container runtime was available in the development environment.